

Project Milestone #3

1 Overview

In Milestone 3, you will extend your Milestone 2 `University/Course/Student` system with features that connect to the next modules: Divide and conquer and Mapping. Each course will now have a prerequisite that should be fulfilled by the student before enrolling to a course.

We will now mainly focus of improving the time complexity of our code by choosing better data structures and better sorting algorithms.

2 Learning Goals

- Implement a hashmap and use it to store the pre-requisite information.
- Improve the sorting algorithms from quadratic sorting to logarithmic sorting.
- Write meaningful unit tests using `unittest` and explain time complexity.

3 Provided Data Files (Milestone 3)

You will be provided with a dataset that contains pre-requisite information for each course.

- `cse.prerequisites.csv` - dataset containing pre-requisite requirement for each course.

4 What You Should Modify vs. What You Should Add

- **Reuse (from Milestone 2):** `Student`, `Course`, `University`.
- **Add:** `HashMap` with separate chaining and rehashing.
- **Modify:**
 - `Course`: add `prerequisite` field (hashmap) and load data from `cse.prerequisites.csv` through your previous data loading mechanisms.

5 What You Must Build

Your system should now check for the prerequisites before enrolling a student into a course.

You should check if the student has previously taken the pre-requisite course from the `course (hashmap)` field from the student class and then `enroll/ waitlist` the student.

6 Tasks and Requirements

6.1 Task 1 — HashMap

Implement a HashMap with the following features.

- Collisions should be handled using separate chaining

- Once the load factor reaches 80%, increase the number of buckets and rehash the table. For example, if the number of keys in the hash table is 4 and the size of the table is 5, then the load factor is $4/5 = 0.8$. At that point, you should double the size of the table. In this example, after rehashing, the new hash table size should be 10.

6.2 Task 2 — Pre-requisite verification

Update the `Course` class with the following changes.

- Add a new `prerequisite` field that is an instance of the `HashMap` and will store all the prerequisites for each course in `cse_prerequisites.csv`
- Change all the sorting algorithms previously used to merge sort and quick sort; the sorting algorithms should still be able to work for name, id, date.
- `request_enroll` should now check the student `course` field to make sure that student has fulfilled the prerequisite requirements and if true, then the student can be added to `enrolled/waitlist`, if false, then your code should raise an exception.

6.3 Task 3 — Unit tests (`unittest`)

Write unit tests to demonstrate correctness. Include a test file named `test_milestone3.py`.

- **HashMap tests:** Test collision handling and rehashing .
- **Enrollment tests:** test if your code checks for pre-requisites.
- **Sorting tests:** roster correctly sorted by id, name, and date for Merge sort and Quick sort.

6.4 Task 4 — Complexity reflection (short report)

Submit a short file named `milestone3_report.txt` (about $\frac{1}{2}$ page) that answers the below with explanations:

- Give Big-*O* time complexity for `put` and `get` in your `HashMap`.
- Give Big-*O* time complexity for each sorting algorithm you implemented.
- Briefly compare your two sorting algorithms: when might one be preferred?

7 Submission Checklist

- All Python source files for Milestone 3 (including `HashMap`).
- `test_milestone3.py` with passing `unittest` cases.
- `milestone3_report.txt` with complexity discussion.
- A short demo script (optional but recommended): `demo_milestone3.py` that shows all the previous demos from previous milestones along with enrolling with pre-requisites.

8 Completion Checkpoints (Lab Support)

During each lab, your TA will do a quick progress check based on the checkpoint items below. Come prepared to show your code running (and tests if applicable) so the TA can verify your completion.

- **Lab — 4/15 (HashMap)**

- Implement the `HashMap`
- Test suite for `HashMap`

TA check: Run tests successfully.

- **Lab — 4/22 (Pre-requisite check for enrollment)**

- Implement the pre-requisite check
- Test suite for enrollment including the pre-requisite checks

TA check: Run tests successfully.

- **Lab — 4/29 (Submission Day: Final Packaging)**

- All required functionality is complete and stable.
- The complexity report is complete, and the submission is packaged correctly.

TA check: Run a short demo showing the full workflow.

9 Grading Rubric (100 points)

Category	What we look for	Points
HashMap core structure	Correct bucket structure; supports put/get operations correctly.	15
Collision handling (separate chaining)	Collisions handled correctly using separate chaining; multiple keys per bucket work correctly.	10
Load factor & rehashing (0.8 threshold)	Rehash triggers at 80% load factor; table size doubles; all keys preserved after rehash.	10
Prerequisite data loading	<code>cse_prerequisites.csv</code> loaded correctly; prerequisites stored in Course HashMap.	10
Prerequisite verification in enrollment	Enrollment correctly checks student history; unmet prerequisite raises exception; valid cases proceed to enroll/waitlist.	15
Merge Sort implementation	Merge sort implemented correctly and used for roster sorting.	10
Quick Sort implementation	Quick sort implemented correctly and used for roster sorting.	10
Unit tests (<code>unittest</code>)	Meaningful tests for: collisions + rehashing, prerequisite checks, and sorting with both algorithms; tests pass.	5
Complexity report	Correct Big-O analysis for HashMap put/get and both sorting algorithms; includes comparison of Merge vs Quick sort.	5
Peer Evaluation	Will be evaluated after milestone submission	10

10 Academic Integrity and Collaboration

Follow the course collaboration policy. You may discuss ideas with classmates, but all code you submit must be written by your group. Cite any external resources used (documentation, tutorials) in comments or in your report.